

BE PROJECT SEMESTER VII REPORT
for
CHECKMATE: SMART PLAGIARISM & AI DETECTOR

Jovita Furtado
Suhani Salgaonkar
Shrain Dhavjekar
Sohail Agha

BACHELOR OF ENGINEERING
IN
COMPUTER ENGINEERING

OF GOA UNIVERSITY



2025-2026

COMPUTER ENGINEERING
PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722

**PADRE CONCEICAO COLLEGE OF ENGINEERING
VERNA GOA – 403722**



BE PROJECT SEMESTER VII REPORT

for

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

Jovita Furtado

202208341

Shrain Dhavjekar

202208371

Suhani Salgaonkar

202208325

Sohail Agha

202208354

Submitted as a requirement for Semester VII Project examination.

2025-2026

Approved By:

.....
Ms. Ramita Karpe

Guide

.....
Dr. Supriya Patil

Head of the Department

.....
Dr. Mahesh Parappagoudar

Principal

Examined By:

Examiner 1:

(Name / Signature / Date)

Examiner 2:

(Name / Signature / Date)

CONTENTS

CHAPTER	Page No.
Acknowledgement.....	i
Abstract.....	ii
List of Figures.....	iii
List of Tables.....	iv
List of Symbols, Abbreviations or Nomenclature	v
1. INTRODUCTION	
1.1 Purpose.....	9
1.2 Project Scope.....	9
1.3 Literature Survey.....	10
2. OVERALL DESCRIPTION	
2.1 Project Perspective.....	11
2.2 Project Functions.....	11
2.3 User Classes and Characteristics.....	12
2.4 Operating Environment.....	12
3. REQUIREMENTS	
3.1 Functional Requirements.....	13
3.2 Non-Functional Requirements.....	13
3.3 Hardware Requirements.....	14
3.4 Software Requirements.....	14
4. SYSTEM DESIGN	
4.1 System Architecture.....	15
4.2 Data Flow Diagram (DFD).....	15
4.3 Module Description.....	16
4.4 AI Model Integration.....	16
5. IMPLEMENTATION AND TESTING	
5.1 Copied Content Detection Model.....	17
5.2 Paraphrase Detection Model.....	18
5.3 AI-Generated Content Detection.....	18
5.4 Originality Scoring Engine.....	18
5.5 Application Features Implemented.....	19
CONCLUSION	23
REFERENCES	24

ACKNOWLEDGEMENT

We express our sincere gratitude to Padre Conceição College of Engineering, Verna, for providing us with the opportunity, resources, and institutional support required to carry out our project titled “CheckMate – AI Powered Plagiarism & Idea Originality Checker.” The encouragement to work on practical, impactful, and academically relevant engineering solutions has played a significant role in shaping the direction and quality of our work.

We extend our heartfelt thanks to our Principal, Dr. Mahesh B. Parappagoudar, for fostering an academic environment that promotes innovation, technical exploration, and project-oriented learning. His vision for academic excellence and holistic student development has been a constant source of motivation throughout the course of this project.

We are deeply grateful to our Head of the Department of Computer Engineering, Dr. Supriya A. Patil, for her continuous guidance, encouragement, and departmental support. Her leadership and emphasis on technical rigor, ethical practices, and professional discipline have helped us maintain clarity, consistency, and purpose in our project development.

We sincerely thank our project guide, Ms. Ramita Karpe, for her invaluable guidance, patient supervision, and constructive suggestions at every stage of the project. Her technical expertise, attention to detail, and continuous feedback greatly contributed to the improvement, refinement, and successful completion of this project.

Finally, we also extend our gratitude to our Review Team Members for their thoughtful evaluations, critical observations, and valuable recommendations. Their insights enabled us to identify areas of improvement and significantly enhanced the overall structure, technical depth, and quality of the project.

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

ABSTRACT

CheckMate is a web-based academic integrity platform designed to analyze research papers for copied content, paraphrased material, and AI-generated writing. The system addresses a practical challenge in higher education: traditional plagiarism tools can identify near-verbatim reuse, but they often fail when a student rewords the source material or relies on generative AI to create fluent but unoriginal text. CheckMate tackles this by combining three complementary detectors so that the weaknesses of one method are covered by the strengths of another.

The copied-content detector uses TF-IDF with cosine similarity to compare the uploaded document against a locally maintained corpus of research papers and reference material. The paraphrase detector uses Sentence-BERT embeddings, which encode meaning at the sentence level and allow semantically similar statements to be matched even when the wording is different. The AI-generated content detector uses a Logistic Regression model trained on labeled human and AI text samples. These three signals are combined through a weighted originality score that makes the output easy to interpret and useful for faculty review.

The prototype is implemented in Flask and supports PDF upload, automatic text extraction with PyMuPDF, sentence tokenization, score visualization, and detailed sentence-level highlighting. Faculty users can adjust the model weights, inspect uploaded files, browse the reference corpus, and manage imported arXiv papers. The result is a flexible and explainable tool that can support plagiarism checking, content review, and classroom-level academic integrity workflows.

Keywords: plagiarism detection, semantic similarity, AI-generated text, TF-IDF, Sentence-BERT, logistic regression, research papers, academic integrity

LIST OF FIGURES

Figure Name	Page No.
Fig. 1 System Architecture	9
Fig. 2 PDF Processing Workflow	10
Fig. 3 Algorithm Score Visualization	13
Fig. 4 Sentence-Level Highlighting View	14
Fig. 5 AI Detection Confusion Matrix	15

LIST OF TABLES

Table No.	Name	Page No.
Table 1	Technology Stack	11
Table 2	Core Modules and Functions	11
Table 3	Default Thresholds and Weights	12
Table 4	Model Evaluation Summary	15

LIST OF ABBREVIATIONS

AI Artificial Intelligence

API Application Programming Interface

BERT Bidirectional Encoder Representations from Transformers

CPU Central Processing Unit

LR Logistic Regression

NLP Natural Language Processing

PDF Portable Document Format

SBERT Sentence-BERT

TF-IDF Term Frequency - Inverse Document Frequency

1. INTRODUCTION

Academic writing is expected to represent original thought, proper citation, and transparent use of source material. In practice, however, the boundary between acceptable reference usage and academic misconduct has become harder to enforce because source material can now be rephrased automatically by paraphrasing tools or generated from scratch by large language models. A system that checks only for exact matching is no longer sufficient for modern academic integrity workflows.

CheckMate was developed to provide a more complete response to this problem. Instead of relying on one detector, it brings together three different views of the same document: lexical overlap, semantic similarity, and AI-writing probability. Each view captures a different kind of risk. Lexical overlap is strong evidence of copied text, semantic similarity is useful for paraphrase detection, and AI probability is useful when the text is syntactically polished but stylistically machine-like.

The system is intended for PDF research papers because that format is common in university submissions and conference-style reports. Once a PDF is uploaded, the system extracts text, removes reference noise, splits the content into sentences, and runs the analysis pipeline. The final interface shows a unified originality score and color-coded sentence highlights so that users can quickly identify where a submission requires revision.

1.1 Purpose

The purpose of the project is to create an explainable detection tool that can identify plagiarism-related issues without reducing the analysis to a black-box score. By showing which sentences were flagged and why, the system gives reviewers and students a more actionable result.

1.2 Project Scope

The project covers document upload, text extraction, corpus matching, paraphrase detection, AI-generated text detection, score aggregation, visual reporting, and corpus expansion through imported arXiv papers. The current implementation is a prototype, but the architecture can be extended for larger institutional deployments.

1.3 Major Contributions

- ? A hybrid analysis pipeline that combines TF-IDF, SBERT, and Logistic Regression.
- ? Sentence-level visualization that identifies which parts of the document triggered which detector.
- ? Faculty-adjustable model weights to reflect local policy or assessment priorities.
- ? A corpus expansion workflow that imports research papers from arXiv and rebuilds the semantic dataset.

2. LITERATURE SURVEY

2.1 Traditional plagiarism detection

Early plagiarism detection systems depended on text fingerprinting, string matching, and n-gram overlap. These methods are efficient when the copied source appears nearly unchanged, but they degrade rapidly when punctuation, word order, or vocabulary changes. TF-IDF with cosine similarity remains valuable because it offers a lightweight and interpretable method for direct match detection.

2.2 Semantic similarity detection

Semantic similarity methods were introduced to address the paraphrase problem. Word embeddings, document embeddings, and transformer-based sentence embeddings allow two passages to be compared at the meaning level rather than the surface-text level. Sentence-BERT is particularly useful in practical applications because it produces embeddings quickly enough for sentence-by-sentence analysis.

2.3 AI-generated text detection

Detecting AI-generated content is a newer research area. Several approaches analyze token likelihood, perplexity, stylometric features, and classifier outputs. In a prototype environment, a TF-IDF plus Logistic Regression model remains attractive because it is simple to train, easy to explain, and fast enough for web-based scoring.

2.4 Hybrid systems

Research increasingly supports hybrid detection systems that combine lexical and semantic methods. This is because plagiarism can appear in multiple forms: copy-paste reuse, paraphrased reuse, and AI-assisted composition. A multi-model design is therefore more robust than any single detector used alone.

2.5 Research gap addressed by CheckMate

- ? A single similarity detector does not reliably identify all types of unoriginal writing.
- ? Most classroom tools do not give sentence-level explanations for why a document was flagged.
- ? Faculty often need adjustable detection sensitivity rather than fixed thresholds.
- ? Many systems do not integrate AI-generated text detection into plagiarism review.

3. SYSTEM ARCHITECTURE

CheckMate follows a layered design that separates user interaction, business logic, model inference, and data storage. The frontend layer provides upload forms, dashboards, and result visualizations. The application layer manages authentication, routing, and report download. The analysis layer performs the core detection tasks, and the data layer stores the reference corpus, categories, and training files.

3.1 Frontend layer

The user interface presents a student dashboard for document analysis and a faculty dashboard for configuration and dataset management. Results are shown in a chart, a circular originality gauge, and a sentence-highlighting area so that the output is easy to interpret.

3.2 Application layer

The Flask server is responsible for login, session handling, upload processing, and API endpoints that expose the datasets and upload history. It also manages faculty-only functions such as updating weights and importing new reference papers.

3.3 Analysis layer

The analysis layer runs three independent models. The TF-IDF model detects copied content through vector similarity. The SBERT model detects paraphrased text using semantic embeddings. The AI detector classifies text as human or AI-generated based on learned patterns in TF-IDF space.

3.4 Data layer

The data layer contains a large corpus of text files, SBERT reference pairs, and AI-labeled training data. The corpus can be expanded by importing arXiv papers, and the semantic dataset is rebuilt automatically from the sentence-level text extracted from the corpus.

3.5 Processing workflow

- ? User uploads a PDF document through the dashboard.
- ? PyMuPDF extracts and cleans the raw text.
- ? The text is split into sentences for granular analysis.
- ? Each sentence is passed through the TF-IDF, SBERT, and AI detectors.
- ? The three scores are combined into a weighted originality score.
- ? The interface highlights flagged sentences and offers a downloadable report.

4. METHODOLOGY AND IMPLEMENTATION

4.1 Preprocessing

Preprocessing is critical because the three detectors work on sentence text that should be as clean and consistent as possible. The TF-IDF path removes stopwords, digits, and punctuation, then lemmatizes terms to reduce variation. The SBERT and AI paths keep the sentence-level structure but still rely on cleaned text from the extraction pipeline.

4.2 TF-IDF copied content model

The copied-content model transforms each cleaned sentence into a vector and compares it with the reference corpus using cosine similarity. A threshold of 0.75 is used because the preprocessing is aggressive and reduces surface noise. If the best match for a sentence exceeds the threshold, the sentence is flagged as copied.

4.3 SBERT paraphrase model

The paraphrase model uses the all-MiniLM-L6-v2 Sentence-BERT encoder. This model returns a 384-dimensional embedding for each sentence, enabling semantic comparison across the uploaded paper and the reference dataset. A similarity threshold of 0.85 is used to capture meaning-preserving rewrites while avoiding overly broad matches.

4.4 AI-generated text model

The AI detector is trained on a labeled CSV file containing text samples and binary labels. TF-IDF features are generated from the training data, and a Logistic Regression classifier is fit on those features. At inference time, each sentence receives an AI probability. A score above 0.5 indicates likely machine-generated writing.

4.5 Weighted scoring engine

The final originality score is computed as 1 minus the weighted sum of the three detector scores. By default, copied content contributes 40 percent of the penalty, paraphrased content contributes 30 percent, and AI-generated content contributes 30 percent. The faculty can change these weights to reflect institutional policy, assessment context, or the type of submission being reviewed.

4.6 Implementation details

- ? Uploaded files are stored in a userinput directory for processing.
- ? Reference papers are automatically organized into categories for browsing.
- ? The TF-IDF corpus is cached after the first load to reduce repeated file I/O.
- ? SBERT embeddings are cached so that repeated analysis is faster.
- ? The AI model is trained once and then reused for later predictions.

4.7 Technology stack

Component	Technology	Purpose
Web framework	Flask	Routing, sessions, and HTML rendering
PDF extraction	PyMuPDF	Extracting text from uploaded papers
Text processing	NLTK and spaCy	Sentence tokenization and lemmatization
Similarity search	scikit-learn	TF-IDF vectorization and cosine similarity
Semantic detection	sentence-transformers	SBERT embeddings
AI detection	Logistic Regression	Sentence-level AI probability

5. RESULTS AND DISCUSSION

The project evaluation demonstrates that the three detectors address different problems with different strengths. TF-IDF is best for direct overlap, SBERT is best for paraphrase detection, and Logistic Regression provides a lightweight but effective AI-writing detector. When combined, the three signals provide a more complete view of originality than any single score could provide.

5.1 Model summary

Model	Accuracy	Comment
AI Detection	90.51%	Strong recall on AI-generated text
TF-IDF Copied Detection	100.00%	Perfect on benchmark copied sentences
SBERT Paraphrase Detection	100.00%	Perfect on benchmark semantic matches

5.2 Interpretation

The results suggest that sentence-level preprocessing and thresholding are effective for the current dataset size. The TF-IDF and SBERT paths are highly reliable when the reference corpus contains the relevant source text, while the AI detector adds coverage for content that does not correspond to a specific source document.

5.3 User-facing feedback

- ? The vertical bar chart shows the relative contribution of each detector.
- ? The circular originality gauge provides an immediate overall score.
- ? Three text columns help users see which sentences were flagged by which model.
- ? A verdict banner classifies the submission as approved or needing revision.

5.4 Practical considerations

The model is intentionally cached in memory so that repeated analyses remain fast enough for interactive use. The main practical limitation is corpus dependence: if the suspicious source does not exist in the local reference set, lexical and semantic match scores may remain low even when the text was borrowed from an external source. The AI detector also works best at sentence level and can be noisy on very short or highly technical sentences.

6. CONCLUSION

CheckMate demonstrates that plagiarism and AI-generated content can be handled in a more complete way by combining lexical matching, semantic matching, and machine-learning classification. The system is not merely a detector but a review tool: it tells the user what was flagged, where it was flagged, and which model contributed to the decision.

The prototype is well suited for academic research papers and can be extended into a broader institutional workflow. Its modular structure makes it straightforward to replace one detector, adjust the scoring policy, or expand the reference corpus without redesigning the entire application.

6.1 Future work

- ? Replace Logistic Regression with a transformer-based AI detector for better robustness.
- ? Extend the system to multilingual plagiarism and paraphrase detection.
- ? Add persistent database storage for reports, uploaded papers, and corpus metadata.
- ? Improve citation-aware handling so properly referenced passages are not flagged unfairly.
- ? Add batch analysis for processing multiple submissions at once.
- ? Integrate internet-scale academic search APIs beyond the local corpus.

REFERENCES

1. C. Park, "In other (people's) words: Plagiarism by university students -- literature and lessons," *Assessment & Evaluation in Higher Education*, 2003.
2. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," *EMNLP-IJCNLP*, 2019.
3. E. Mitchell et al., "DetectGPT: Zero-shot machine-generated text detection using probability curvature," *ICML*, 2023.
4. S. Gehrmann, H. Strobelt, and A. Rush, "GLTR: Statistical detection and visualization of generated text," *ACL System Demonstrations*, 2019.
5. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, 2011.
6. S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.